Lab 2 • Part 3
Calibration-Free NF4 PTQ

學號：314551126 日期：2026-10-28 模型：Qwen/Qwen3-4B

本實驗採作業允許的替代 PTQ 路線：固定 NF4 權重量化，使用 fake quantization 模擬數值。量化只讀原始權重，不需要 calibration dataset、activation 統計或目標模型的設定搜尋。

1 結果與評分門檻

設定	WikiText-2 PPL	來源
原始 FP16	13.6378	本次獨立控制實驗
固定 NF4 W4A16	14.6063	本次量化後的官方評估
AdaptQuant 比較值	13.98	Lab 2.md 提供的評分基準

NF4 相對 FP16 的 PPL 差為 +0.9685，相對作業比較值為 +0.6263。依作業評分表，PPL ≤15 對應性能項15分；本次結果對應 15 分。報告項的5分仍由助教評閱。

2 固定實驗設定

項目	設定
權重／activation	NF4 4-bit 數值／原本 FP16 流程
量化範圍	全部252個 decoder Linear；不做逐層精度配置
未量化範圍	共享 input embedding／lm_head 保留 FP16；其他原始非 Linear 元件不變
Block／scale	每64個權重一組；直接取該組 FP32 absmax
格式選擇	固定16個 NF4 codepoints；不做 clipping search、double quant 或 mixed precision
執行環境	Python 3.11.17；torch 2.5.1+cu121；Tesla T4

規則在評估前固定。FP16 控制實驗與 test PPL 都不會回流量化器，亦不使用 Part 2 的 GPTQ、SVD、剪枝 checkpoint 或 calibration 統計。

方法與 calibration-free 性質

3 NF4 映射

NF4 使用預先定義的非均勻16值 codebook，源自 QLoRA 的 NormalFloat 格式。本實驗只實作此格式的量化映射，沒有 LoRA、optimizer 或 fine-tuning。

對每一組64個權重，依序計算：
1. s = max(abs(w))。
2. u = w / s。
3. 選擇與 u 最近的固定 NF4 codepoint c。
4. 重建 w_hat = s * c，再轉回 FP16 作為 Linear 權重。
全零 block 以 scale=0、code=7 重建精確的零；邊界距離相等時固定選較低的 code。

Code index	固定值
0–7	-1.000000, -0.696193, -0.525073, -0.394917, -0.284441, -0.184773, -0.091050, 0.000000
8–15	0.079580, 0.160930, 0.246112, 0.337915, 0.440710, 0.562617, 0.722957, 1.000000

Absmax 是既定映射直接讀取權重的結果，不是以模型輸出或校正樣本搜尋的參數。Codebook 不是從 Qwen3 權重分布訓練出來；block size、層範圍與精度皆事先固定。

4 資料與程序隔離

nf4.py 只依賴 PyTorch，函式輸入是權重，沒有 dataset 或 dataloader。量化期間對模型與所有子模組安裝禁止 forward 的 hook；量化完成後才移除。程式到正式評估階段才匯入 evaluate.py 並讀取 WikiText-2 test。

量化後使用固定的8個 token 驗證儲存／重載的 logits；此檢查只驗證序列化一致性，不調整任何量化參數。每層 weight MSE 只作格式診斷，不用來選層、選精度或搜尋 clipping。

5 Fake quantization 與實際儲存

允許的 fake quantization 把 NF4 重建數值放回原始 FP16 tensor。推論仍用原始 PyTorch Linear 的 FP16 計算流程。Checkpoint 是 FP16 儲存，不能據此宣稱實際4-bit檔案壓縮或 Blackwell 加速；本實驗驗證的是量化數值與 PPL。

量化範圍共 3,633,315,840 個權重，整體權重重建 MSE=4.7352667e-06。Embedding／lm_head 的共享與原始值在量化過程保持不變。

驗證、觀察與方法差異

6 數值驗證

五項測試涵蓋：精確 codepoints／零 block、與獨立最近鄰 oracle 比對、尾端 padding 與執行 chunk 不變性、固定量化範圍與共享 head 保護，以及 bitsandbytes 0.48.1 的 NF4 對照。對照關閉 double quant，重建 tensor 完全一致。

完整模型保存後重載，固定輸入的 logits bitwise 相同，embedding／lm_head 仍共享同一個參數。正式 PPL 完全透過提供的 run_eval.py 與 evaluate.eval_ppl_wikitext2()，sequence length 2048，原檔 SHA-256 記錄於結果 JSON。

7 結果觀察與限制

本次固定 NF4 的 PPL 為 14.6063，FP16 為 13.6378。差異反映低精度權重重建造成的數值擾動；沒有使用訓練或輸出誤差補償來恢復品質。部分層的權重 MSE 較大，但本方法仍使用相同固定格式，沒有針對這些層另選較高精度。

主要實作要點是固定 codebook 的最近值選擇、全零 block、最後不足64個元素的處理、FP32 scale 與 FP16 重建的精度順序。Chunk 只影響暫存記憶體，因 Linear 每列長度可被64整除，改變 chunk 不會改變 block 邊界或格式。

限制包括：只量化 decoder 權重、activation 保持原 FP16 流程、共享 embedding／head 保留 FP16，且沒有真實4-bit儲存或專用低精度 matmul。若後續研究其他格式，必須維持無校正資料與無目標模型精度搜尋的條件。

後續可在不改固定映射的條件下實作 packed 儲存與專用 kernel，減少檔案大小與計算成本；品質與速度仍需要另做驗證。

8 與 AdaptQuant 的重要差異

本實驗是作業允許的其他 PTQ 方法，並非 AdaptQuant 重現。AdaptQuant 題名為 per-tile MX format selection；本實驗使用固定 NF4 權重格式，不實作其每 tile MX 選擇規則或 NVIDIA Blackwell kernel。13.98 僅作為作業提供的比較值，不是本次重現其程式得到的結果。

本次未取得可讀的 AdaptQuant 全文／原始實作，因此不推測其未確認的 bit 配置、activation 格式或閾值；與其完整量化設定的可比性有限。作業明定不論方法，都以相同 Qwen3-4B 和 PPL 分級評分。

重現步驟與來源

9 從乾淨環境重現

需求：Python 3.11、單張 T4 等級 CUDA GPU。首次需要網路下載 Qwen/Qwen3-4B 與 WikiText-2；已下載可設 HF_HUB_OFFLINE=1、HF_DATASETS_OFFLINE=1。模型 commit 固定記錄於 artifacts/source.json。

在 Part3 目錄執行：bash run.sh
入口會自動建立本地 .venv 並安裝依賴。
若已有符合版本的環境：
PART3_PYTHON=/absolute/path/to/python bash run.sh

run.sh 會安裝依賴、重跑 FP16 控制實驗、建立固定 NF4 模型、保存／重載、跑官方 PPL，最後產生本 PDF 與 Markdown。
若 checkpoint 已存在，可只執行：
python run_part3.py --mode nf4 --eval-only
python make_report.py

檔案	內容
nf4.py	固定 codebook、absmax 量化與反量化
run_part3.py	模型建置、序列化驗證、官方評估包裝
test_nf4.py	五項獨立數值與範圍測試
results/fp16_result.json、nf4_result.json	實測 PPL、來源 commit、evaluator SHA-256
artifacts/nf4/manifest.json	固定 policy、量化範圍、權重誤差、重載結果
artifacts/nf4/model/	FP16 fake-quantized checkpoint，非 packed NF4

Model revision：1cfa9a7208912126459214e8b04321603b3df60c
Policy SHA-256：818fabdcb639e9d9ba85b5a3bf497884c96587074f5269ae9b87cdf0f6badd77

10 參考來源

[1] Dettmers et al., QLoRA: Efficient Finetuning of Quantized LLMs. 本實驗使用其 NF4 格式，不使用其 fine-tuning 方法。

https://arxiv.org/abs/2305.14314

[2] bitsandbytes 0.48.1，functional.get_4bit_type("nf4") 與 quantize_4bit/dequantize_4bit，作為固定常數及獨立數值對照。
https://github.com/bitsandbytes-foundation/bitsandbytes

[3] Lab 2.md，Part 3 的 calibration-free 條件、fake quantization 許可、AdaptQuant 比較值與評分表。
題目提供的論文連結：https://openreview.net/forum?id=ZGeR4gkEZq